

# Digital Electronics

A full student textbook for number systems, Boolean algebra, K-Maps, logic families, combinational circuits, flip-flops, registers, counters and semiconductor memories—built from the official model-paper scope and examination pattern.

**COURSE CODE****3044****SEMESTER****Third Semester****PROGRAMME****Common to BM / EC / EL****SCHEME****Revision 2021****MAXIMUM MARKS****75****TIME****3 Hours****CREDITS / CATEGORY****Not specified in supplied source****PRIMARY SOURCE****Official Model Question Paper, Sets 1 & 2**[Start Learning](#)[Download Lesson PDF](#)**FOUR RECONSTRUCTED LEARNING MODULES**

## From binary digits to complete digital systems

The handbook follows the topic scope and module-outcome codes evidenced in the supplied official-style model paper. Reconstructed titles are clearly identified as handbook organization, not claimed as official syllabus titles.

**Module 1**

Number systems, gates, Boolean algebra, K-Maps

**Module 2**

Logic families and combinational circuits

## COURSE OVERVIEW

# What Digital Electronics is and why it matters

Digital electronics represents and processes information using discrete logic levels, normally 0 and 1. It is the foundation of computers, embedded systems, communication equipment, medical instruments, industrial control and modern consumer electronics.

## Information as bits

A bit has two states. Groups of bits represent numbers, instructions, characters, addresses and control information.

## Logic as decisions

Logic gates implement decisions such as AND, OR, NOT and XOR. Boolean algebra gives a mathematical language to simplify those decisions.

## Memory and sequence

Flip-flops and registers remember past information. Counters and state circuits create controlled sequences over time.

## Importance and applications

### Computing

Processors, memories, storage interfaces and digital buses.

### Communication

Encoding, multiplexing, switching, timing and control.

### Automation

Programmable controllers, counters, sensors and machine sequencing.

Medical and instrumentation

Digital measurement, data acquisition, alarms and display systems.

Source limitation

The supplied document is a model question paper, not a complete syllabus. Therefore, credits, category, course objectives, exact CO-PO mapping, periods and official reference books are not fabricated. Module titles below are a logical handbook reconstruction from the questions and module-outcome codes.

STUDY METHOD

How to use this handbook

For every topic: understand the idea, inspect the truth table or diagram, work the solved example, then attempt the practice question without looking at the answer.

1. Learn the rule

Read definitions, formulas and design rules.

2. Trace the logic

Follow signal paths, state transitions and timing diagrams.

3. Solve step-by-step

Show all conversions, K-Map groups and design tables.

4. Practise for marks

Use one-mark, three-mark and seven-mark patterns.

TABLE OF CONTENTS

Clickable handbook map

All links point to a valid section in this standalone file.

Source & Exam Information



Learning Roadmap



|                        |   |
|------------------------|---|
| Module 1               | → |
| Module 2               | → |
| Module 3               | → |
| Module 4               | → |
| Rule and Formula Banks | → |
| Solved Examples        | → |
| Expected Questions     | → |
| Practice Section       | → |
| Quick Revision         | → |
| Fresh Model Paper      | → |
| Complete Model Answers | → |

SOURCE AND EXAMINATION INFORMATION

# Confirmed examination pattern

The two supplied model sets establish the title, semester, programme applicability, module-outcome codes and paper structure.

| Item                             | Confirmed information                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------|
| Subject                          | Digital Electronics                                                                       |
| Course code                      | 3044                                                                                      |
| Semester                         | Third Semester                                                                            |
| Programme                        | Common to BM / EC / EL                                                                    |
| Scheme                           | Revision 2021                                                                             |
| Time                             | 3 hours                                                                                   |
| Maximum marks                    | 75                                                                                        |
| Part A                           | 9 questions × 1 mark = 9; answer all                                                      |
| Part B                           | 10 questions; answer any 8; 3 marks each = 24                                             |
| Part C                           | 6 compulsory groups, each with internal OR; 7 marks per group = 42                        |
| Available module-outcome codes   | M1.01, M1.03, M1.04, M2.02, M2.04, M3.01, M3.02, M3.03, M3.04, M4.01, M4.02, M4.03, M4.04 |
| Not specified in supplied source | Credits, category, periods, formal objectives, CO-PO mapping, official reference books    |

# Four reconstructed modules

The following module titles are a handbook organization inferred from the topic distribution in the model papers.

| Module                                                     | Evidenced outcome codes    | Core topics                                                                                   | Exam emphasis                                                    |
|------------------------------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 1. Number Systems, Logic Gates, Boolean Algebra and K-Maps | M1.01, M1.03, M1.04        | Conversions, complements, binary arithmetic, universal gates, Boolean laws, De Morgan, K-Maps | Short calculations and 7-mark conversion / minimization problems |
| 2. Logic Families and Combinational Logic Circuits         | M2.02, M2.04               | CMOS/ECL parameters, adders, multiplexers, demultiplexers, binary-to-Gray conversion          | 3-mark features and 7-mark truth-table/K-Map designs             |
| 3. Flip-Flops, Shift Registers, Ring and Johnson Counters  | M3.01, M3.02, M3.03, M3.04 | Sequential logic, flip-flops, conversions, register types, ring and Johnson sequence          | Symbols, truth tables, diagrams and conversion/design questions  |
| 4. Counters and Semiconductor Memories                     | M4.01, M4.02, M4.03, M4.04 | Ripple/synchronous counters, MOD design, timing diagrams, RAM/ROM/PROM/EPROM/EEPROM           | Counter design and memory comparisons                            |

MODULE 1 • M1.01 / M1.03 / M1.04

## Number Systems, Logic Gates, Boolean Algebra and K-Maps

This title is reconstructed from the official model-paper topics. The chapter develops the complete calculation and logic-design foundation required by those questions.

### Learning targets

- Convert between decimal, binary and hexadecimal—including fractions.
- Perform binary arithmetic and complement subtraction.
- Use logic gates and universal-gate implementation.
- Simplify expressions using Boolean laws and K-Maps.

### Core exam skills

- Show each conversion step.
- Use the specified bit width for complements.
- Draw symbols and truth tables neatly.
- Mark K-Map groups and write final reduced expression.

## 1. Number systems and positional value

A number system is defined by its radix or base. In a positional number system, the value of a digit depends on both the digit and its position.

$$(d_n...d_1d_0.d_{-1}d_{-2})_r = \sum d_i r^i$$

| System      | Base | Digits   | Example                 |
|-------------|------|----------|-------------------------|
| Decimal     | 10   | 0–9      | (125.4) <sub>10</sub>   |
| Binary      | 2    | 0,1      | (10101.01) <sub>2</sub> |
| Hexadecimal | 16   | 0–9, A–F | (4BAC) <sub>16</sub>    |

### Place-value chart

| Position         | ... | 3              | 2              | 1              | 0              | -1              | -2              |
|------------------|-----|----------------|----------------|----------------|----------------|-----------------|-----------------|
| Weight in base r | ... | r <sup>3</sup> | r <sup>2</sup> | r <sup>1</sup> | r <sup>0</sup> | r <sup>-1</sup> | r <sup>-2</sup> |

### Hexadecimal digit table

| Hex     | A    | B    | C    | D    | E    | F    |
|---------|------|------|------|------|------|------|
| Decimal | 10   | 11   | 12   | 13   | 14   | 15   |
| Binary  | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

**Malayalam Support**  
 Radix എന്നത് base ആണ്. Binary-ൽ ഓരോ position-നും 2ന്റെ power; hexadecimal-ൽ 16ന്റെ power ഉപയോഗിക്കുന്നു.

## 2. Conversion methods

**Binary → Decimal**  
 Multiply each bit by its place value and add.

$$(10101)_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 21$$

**Decimal integer → Binary**  
 Repeatedly divide by 2 and read remainders upward.

$$35_{10} = 100011_2$$

**Decimal fraction → Binary**  
 Repeatedly multiply the fraction by 2 and read integer parts downward.

$$0.625_{10} = .101_2$$

**Hex → Decimal**

Multiply each hexadecimal digit by powers of 16.

$$(AB6)_{16} = 10 \times 16^2 + 11 \times 16 + 6 = 2742_{10}$$

**Hex ↔ Binary**

Replace every hexadecimal digit by four binary bits.

$$(4BAC)_{16} = 0100\ 1011\ 1010\ 1100_2$$

**Decimal → Hex**

Repeatedly divide integer part by 16. Use A–F for remainders 10–15.

$$125_{10} = 7D_{16}$$

**Interactive binary counter**

Use the buttons to observe binary states and decimal value.

0 0 0 0

Decimal: 0

**3. Binary arithmetic**

| Operation      | Rules                                                                   |
|----------------|-------------------------------------------------------------------------|
| Addition       | $0+0=0$ ; $0+1=1$ ; $1+0=1$ ; $1+1=10$ ; $1+1+1=11$                     |
| Subtraction    | $0-0=0$ ; $1-0=1$ ; $1-1=0$ ; $0-1$ requires borrow                     |
| Multiplication | $0 \times \text{anything} = 0$ ; $1 \times \text{bit} = \text{bit}$     |
| Division       | Same long-division idea as decimal, but quotient digits are only 0 or 1 |

**Binary addition example**

$$\begin{array}{r} 101101 \\ + 011011 \\ \hline 1001000 \end{array}$$

$45 + 27 = 72$ , so the result is verified.

## Binary multiplication example

```

1011
× 101
-----
1011
0000
101100
-----
110111

```

$11 \times 5 = 55 = 110111_2$ .

## 4. Signed numbers and complements

One's complement is formed by inverting every bit. Two's complement is one's complement plus 1. In an n-bit signed two's-complement system, the leftmost bit is the sign bit.

One's complement:  $0 \leftrightarrow 1$

Two's complement = One's complement + 1

### Example: 8-bit two's complement of 14

1 14 = 00001110

2 Invert bits → 11110001

3 Add 1 → 11110010

### Subtraction: 46 – 14

```

46 = 00101110
14 = 00001110
2's comp(14) = 11110010

```

```

00101110
+ 11110010
-----
1 00100000

```

Discard the end carry. Result =  $00100000_2 = 32_{10}$ .

### Malayalam Support

Two's complement subtraction-ൽ subtract ചെയ്യേണ്ട number-ന്റെ bits invert ചെയ്ത് 1 കൂട്ടുക. ശേഷം minuend-നോട് add ചെയ്യുക. End carry ഉണ്ടെങ്കിൽ അത് discard ചെയ്യണം.

### Common mistake

Use the required bit width before complementing. Do not complement an unpadded number.



## 5. Logic levels and gates

Course 3044

In positive logic, the higher voltage level represents logic 1 and the lower voltage level represents logic 0. A logic gate performs a Boolean operation on one or more binary inputs.

### Logic-gate symbol guide

AND is D-shaped, OR has a curved input, NOT is triangular, and inversion is shown by a small circle.

#### AND

$$Y = AB$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

#### OR

$$Y = A + B$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

#### NOT

$$Y = \bar{A}$$

| A | Y |
|---|---|
| 0 | 1 |
| 1 | 0 |

NAND

$Y = (AB)^{\overline{\phantom{x}}}$

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

NOR

$Y = (A+B)^{\overline{\phantom{x}}}$

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

XOR

$Y = A \oplus B$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**Gate classification**  
**Basic:** AND, OR, NOT. **Universal:** NAND, NOR. **Special:** XOR, XNOR.

6. Universal gates

A universal gate can implement any Boolean function. NAND and NOR are universal because NOT, AND and OR can be created using only one gate type.

| Function using NAND | Connection                                       | Result                   |
|---------------------|--------------------------------------------------|--------------------------|
| NOT                 | Join both NAND inputs to A                       | A NAND A = $\bar{A}$     |
| AND                 | NAND A,B, then invert output with NAND           | $[(AB)] = AB$            |
| OR                  | Invert A and B using NAND, then NAND the results | $(\bar{A}\bar{B}) = A+B$ |
| XOR                 | Four-NAND standard network                       | $A \oplus B$             |

### Malayalam Support

Universal gate എന്നത് മറ്റെല്ലാ basic gates-ഉം നിർമ്മിക്കാൻ കഴിയുന്ന gate ആണ്. NAND മാത്രം ഉപയോഗിച്ചും NOR മാത്രം ഉപയോഗിച്ചും മുകളവൻ digital circuit നിർമ്മിക്കാം.

## 7. Boolean algebra

| Law          | Expression                          |
|--------------|-------------------------------------|
| Identity     | $A+0=A$ ; $A \cdot 1=A$             |
| Null         | $A+1=1$ ; $A \cdot 0=0$             |
| Idempotent   | $A+A=A$ ; $A \cdot A=A$             |
| Complement   | $A+\bar{A}=1$ ; $A \cdot \bar{A}=0$ |
| Involution   | $(\bar{\bar{A}})=A$                 |
| Commutative  | $A+B=B+A$ ; $AB=BA$                 |
| Associative  | $A+(B+C)=(A+B)+C$ ; $A(BC)=(AB)C$   |
| Distributive | $A(B+C)=AB+AC$ ; $A+BC=(A+B)(A+C)$  |
| Absorption   | $A+AB=A$ ; $A(A+B)=A$               |
| De Morgan 1  | $(A+B)=\bar{A}\bar{B}$              |
| De Morgan 2  | $(AB)=\bar{A}+\bar{B}$              |

### Algebraic simplification example

$$\begin{aligned} Y &= \bar{A}B + AB + A\bar{B} \\ &= B(\bar{A} + A) + A\bar{B} \\ &= B + A\bar{B} \\ &= (B + A)(B + \bar{B}) \\ &= A + B \end{aligned}$$

## 8. SOP, POS, minterms and maxterms

A minterm is an AND term containing every variable once. A maxterm is an OR term containing every variable once. Canonical SOP is a sum of minterms; canonical POS is a product of maxterms.

$$F(A,B,C) = \Sigma m(1,3,5,7)$$

$$F(A,B,C) = \Pi M(0,2,4,6)$$

## 9. Karnaugh Maps

Course 3044

A K-Map arranges adjacent cells in Gray-code order so that one variable changes between neighbouring cells. Groups must contain 1, 2, 4, 8 or 16 cells.

### Two-variable K-Map

| A\B | 0  | 1  |
|-----|----|----|
| 0   | m0 | m1 |
| 1   | m2 | m3 |

### Three-variable K-Map

| A\BC | 00 | 01 | 11 | 10 |
|------|----|----|----|----|
| 0    | m0 | m1 | m3 | m2 |
| 1    | m4 | m5 | m7 | m6 |

### Four-variable K-Map

| AB\CD | 00  | 01  | 11  | 10  |
|-------|-----|-----|-----|-----|
| 00    | m0  | m1  | m3  | m2  |
| 01    | m4  | m5  | m7  | m6  |
| 11    | m12 | m13 | m15 | m14 |
| 10    | m8  | m9  | m11 | m10 |

### Grouping rules

- Use largest possible groups.
- Every required 1 must be covered.
- Overlapping is allowed.
- Wrap-around adjacency is allowed.
- Diagonal cells are not adjacent.

- Don't-care cells may be used only when helpful.

Course 3044

## Typical reduction workflow

- 1 Place all minterms and don't-cares.
- 2 Form essential largest groups.
- 3 Write one product term per group.
- 4 OR the terms and verify coverage.

### Solved K-Map: $F(A,B,C)=\Sigma m(0,2,3,4,5,6)$

Place 1s at  $m_0, m_2, m_3, m_4, m_5, m_6$ .

Group  $m_0, m_2, m_4, m_6$  as a quad  $\rightarrow \bar{C}$ .

Group  $m_4, m_5$  as a pair  $\rightarrow A\bar{B}$ .

Group  $m_2, m_3$  as a pair  $\rightarrow \bar{A}B$ .

Final:  $F = \bar{C} + A\bar{B} + \bar{A}B$ .

### Four-variable example with don't-cares

$F(A,B,C,D)=\Sigma m(1,4,7,10,13)+\Sigma d(5,14,15)$ .

Use d-cells only to enlarge groups:

$m_1, m_5(d), m_{13}, m_?$  cannot form a valid full quad unless every required cell is adjacent.

One possible minimal cover is found by mapping and selecting valid pairs/quads.

Always show the marked map, groups, reduced terms and final SOP. Do not treat every don't-care as mandatory 1.

One mark

- Find decimal value of  $10101_2$ .
- Write 1's or 2's complement.
- Name a universal gate.

Three marks

- Convert a hexadecimal integer/fraction.
- State De Morgan's theorems.
- Reduce a small expression using K-Map.

Seven marks

- Mixed number-system operations.
- Universal-gate implementation.
- Four-variable K-Map with don't-cares.

MODULE 2 • M2.02 / M2.04

Logic Families and Combinational Logic Circuits

This reconstructed chapter covers the logic-family parameters and combinational design problems evidenced in the official model paper.

Learning targets

- Compare TTL, CMOS and ECL.
- Explain fan-out, delay and power dissipation.
- Design adders, multiplexers and demultiplexers.
- Derive a four-bit binary-to-Gray converter.

Combinational principle

Output depends only on present inputs. There is no stored state and normally no clock.

Combinational circuit-ന്റെ output ഇപ്പോഴുള്ള input-നെ മാത്രം ആശ്രയിക്കുന്നു; previous state ഓർക്കില്ല.

1. Logic families and parameters

A logic family is a group of compatible digital ICs built using a common circuit technology and logic-level convention.

|                     |                                                                   |
|---------------------|-------------------------------------------------------------------|
| Fan-in              | Maximum number of inputs a gate is designed to accept             |
| Fan-out             | Number of standard gate inputs one output can drive reliably      |
| Propagation delay   | Time between input transition and corresponding output transition |
| Power dissipation   | Average power consumed by a gate                                  |
| Noise margin        | Maximum unwanted noise voltage tolerated without logic error      |
| Speed-power product | Propagation delay × power dissipation; lower is generally better  |

| Feature              | TTL                | CMOS                                   | ECL                            |
|----------------------|--------------------|----------------------------------------|--------------------------------|
| Device technology    | Bipolar transistor | MOSFET                                 | Bipolar differential switching |
| Power dissipation    | Moderate           | Very low in static condition           | High                           |
| Speed                | Moderate / fast    | Modern CMOS can be very fast           | Traditionally fastest          |
| Noise immunity       | Moderate           | High                                   | Moderate                       |
| Input impedance      | Moderate           | Very high                              | Lower than CMOS                |
| Model-paper emphasis | General comparison | Least power dissipation; CMOS features | Fastest family; ECL features   |

### Malayalam Support

Propagation delay എന്നത് input change ചെയ്തതിനു ശേഷം output change കാണാൻ എടുക്കുന്ന ചെറിയ സമയമാണ്. Fan-out ഒരു output എത്ര gate inputs drive ചെയ്യാമെന്ന് പറയുന്നു.

## 2. Combinational vs sequential circuits

| Feature           | Combinational                | Sequential                        |
|-------------------|------------------------------|-----------------------------------|
| Output depends on | Present inputs only          | Present inputs and previous state |
| Memory            | No                           | Yes                               |
| Clock             | Normally not required        | Usually required                  |
| Examples          | Adder, MUX, DEMUX, converter | Flip-flop, register, counter      |

## 3. Half adder and full adder

### Half adder

$$\text{Sum } S = A \oplus B$$

$$\text{Carry } C = AB$$

| A | B | S | C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

### Full adder

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

| A | B | Cin | S | Cout |
|---|---|-----|---|------|
| 0 | 0 | 0   | 0 | 0    |
| 0 | 0 | 1   | 1 | 0    |
| 0 | 1 | 0   | 1 | 0    |
| 0 | 1 | 1   | 0 | 1    |
| 1 | 0 | 0   | 1 | 0    |
| 1 | 0 | 1   | 0 | 1    |
| 1 | 1 | 0   | 0 | 1    |
| 1 | 1 | 1   | 1 | 1    |

### Full adder using two half adders

HA1 adds A and B. HA2 adds the first sum and Cin. OR the two carry outputs.

## 4. Parallel adder

A multi-bit addition requires one full adder per bit. The carry output of one stage feeds the carry input of the next stage. A four-bit parallel adder adds  $A_3A_2A_1A_0$  and  $B_3B_2B_1B_0$  simultaneously, while carry ripples from LSB to MSB.

### Diagram guidance

Draw the corresponding block or logic diagram with standard labels, signal direction and input/output names. Use the truth table and equations beside this panel to verify the drawing.

## 5. Multiplexers

A multiplexer is a data selector. It selects one of many input lines and connects it to one output according to select lines.



For 2<sup>n</sup> inputs, number of select lines = n

2 inputs → 1 select

4 inputs → 2 selects

8 inputs → 3 selects

16 inputs → 4 selects

## Diagram guidance

Draw the corresponding block or logic diagram with standard labels, signal direction and input/output names. Use the truth table and equations beside this panel to verify the drawing.

$$4:1 \text{ MUX: } Y = I_0 \bar{S}_1 \bar{S}_0 + I_1 \bar{S}_1 S_0 + I_2 S_1 \bar{S}_0 + I_3 S_1 S_0$$

| S1 | S0 | Selected input |
|----|----|----------------|
| 0  | 0  | I0             |
| 0  | 1  | I1             |
| 1  | 0  | I2             |
| 1  | 1  | I3             |

## Interactive 4:1 MUX

Set input bits and select lines. The selected input appears at Y.

S1S0: 00 Y: 0

## Malayalam Support

Multiplexer പല input lines-ൽ നിന്ന് select lines ഉപയോഗിച്ച് ഒരു input മാത്രം output-ലേക്ക് തിരഞ്ഞെടുക്കുന്ന data selector ആണ്.

## 6. Demultiplexers

A demultiplexer is a data distributor. One input is routed to one of several outputs according to select lines.

| S1 | S0 | Active output in 1:4 DEMUX |
|----|----|----------------------------|
| 0  | 0  | Y0 = D                     |
| 0  | 1  | Y1 = D                     |
| 1  | 0  | Y2 = D                     |
| 1  | 1  | Y3 = D                     |

$$Y_0 = D \bar{S}_1 \bar{S}_0; Y_1 = D \bar{S}_1 S_0; Y_2 = D S_1 \bar{S}_0; Y_3 = D S_1 S_0$$

## 7. Binary-to-Gray converter

Gray code changes only one bit between adjacent numbers. This reduces transition ambiguity in encoders and position systems.

$$\text{MSB: } G_3 = B_3$$

Remaining:  $G_2=B_3 \oplus B_2$ ;  $G_1=B_2 \oplus B_1$ ;  $G_0=B_1 \oplus B_0$

| Binary B3B2B1B0 | Gray G3G2G1G0 |
|-----------------|---------------|
| 0000            | 0000          |
| 0001            | 0001          |
| 0010            | 0011          |
| 0011            | 0010          |
| 0100            | 0110          |
| 0101            | 0111          |
| 0110            | 0101          |
| 0111            | 0100          |
| 1000            | 1100          |
| 1001            | 1101          |
| 1010            | 1111          |
| 1011            | 1110          |
| 1100            | 1010          |
| 1101            | 1011          |
| 1110            | 1001          |
| 1111            | 1000          |

Design method

- 1. Prepare Binary-to-Gray conversion table.
- 2. Write each Gray output column as a function of B3,B2,B1,B0.
- 3. Plot each output on a K-Map.
- 4. Reduced results:
  - G3=B3
  - G2=B3⊕B2
  - G1=B2⊕B1
  - G0=B1⊕B0
- 5. Implement using one direct connection and three XOR gates.

One mark

- Fastest logic family.
- Least-power logic family.
- Select lines for 8:1 MUX.

Three marks

- Features of CMOS/ECL.
- Define fan-out and propagation delay.
- Draw D/T or half-adder table.

Seven marks

- Design full adder.
- Explain 1:4 DEMUX.
- Design 4-bit Binary-to-Gray converter.

MODULE 3 • M3.01 / M3.02 / M3.03 / M3.04

# Flip-Flops, Shift Registers, Ring and Johnson Counters

This reconstructed chapter develops the sequential-logic topics repeatedly tested in the official model paper.

## Learning targets

- Distinguish combinational and sequential logic.
- Use SR, JK, D and T flip-flops.
- Convert JK to T and JK to D.
- Trace register, ring and Johnson sequences.

## Sequential principle

Output depends on present input and stored state. A clock coordinates state changes.

Sequential circuit previous state ഓർക്കുന്നു. Clock edge വന്നപ്പോൾ state update ചെയ്യും.

## 1. Clock and state concepts

A clock is a periodic digital waveform. Edge-triggered flip-flops respond at a rising edge or falling edge. Present state is the current stored output; next state is the value after the active clock edge.

## Diagram guidance

Draw the corresponding block or logic diagram with standard labels, signal direction and input/output names. Use the truth table and equations beside this panel to verify the drawing.

## 2. Flip-flop summary

| Flip-flop | Inputs | Key action                       | Important condition                      |
|-----------|--------|----------------------------------|------------------------------------------|
| SR        | S,R    | Set, reset or hold               | $S=R=1$ invalid for active-high NOR form |
| JK        | J,K    | Improved SR; toggle when $J=K=1$ | Avoid race-around with proper triggering |
| D         | D      | Next Q follows D                 | Data storage / delay                     |
| T         | T      | Toggle when $T=1$                | Counters and divide-by-2                 |

### SR truth table

| S | R | $Q(n+1)$ | Meaning   |
|---|---|----------|-----------|
| 0 | 0 | $Q_n$    | Hold      |
| 0 | 1 | 0        | Reset     |
| 1 | 0 | 1        | Set       |
| 1 | 1 | Invalid  | Forbidden |

### JK truth table

| J | K | $Q(n+1)$    |
|---|---|-------------|
| 0 | 0 | $Q_n$       |
| 0 | 1 | 0           |
| 1 | 0 | 1           |
| 1 | 1 | $\bar{Q}_n$ |

### D truth table

| D | $Q(n+1)$ |
|---|----------|
| 0 | 0        |
| 1 | 1        |

$$Q(n+1)=D$$

## T truth table

| T | Q(n+1)         |
|---|----------------|
| 0 | Q <sub>n</sub> |
| 1 | $\bar{Q}_n$    |

$$Q(n+1) = T \oplus Q_n$$

### 3. SR flip-flop using NAND gates

A cross-coupled NAND latch normally uses active-low inputs  $\bar{S}$  and  $\bar{R}$ . Both inputs high hold the state; pulling  $\bar{S}$  low sets Q; pulling  $\bar{R}$  low resets Q. Both low is invalid.

#### Malayalam Support

NAND SR latch-ൽ inputs active-low ആണ്.  $\bar{S}=0$  set,  $\bar{R}=0$  reset. രണ്ടും 0 ആക്കരുത്.

### 4. Flip-flop conversion

Conversion means forcing an available flip-flop to behave like a desired flip-flop. Use the desired next state and the available flip-flop excitation table, then simplify input equations.

#### JK → T

For T=0, hold. For T=1, toggle. A JK flip-flop already toggles for J=K=1.

$$J = T, K = T$$

#### JK → D

To make  $Q(n+1)=D$ :

$$J = D, K = \bar{D}$$

#### Malayalam Support

Flip-flop conversion-ൽ desired flip-flop-ന്റെ next state ആദ്യം എഴുതുക. Available flip-flop ആ state ഉണ്ടാക്കാൻ J,K പോലുള്ള inputs എന്താകണം എന്ന് excitation table ഉപയോഗിച്ച് കണ്ടെത്തുക.

### 5. Shift registers

A shift register is a chain of flip-flops that stores and moves binary data one position per clock pulse.

#### Diagram guidance

Draw the corresponding block or logic diagram with standard labels, signal direction and input/output names. Use the truth table and equations beside this panel to verify the drawing.

| Type | Input    | Output   | Use                           |
|------|----------|----------|-------------------------------|
| SISO | Serial   | Serial   | Delay line                    |
| SIPO | Serial   | Parallel | Serial-to-parallel conversion |
| PISO | Parallel | Serial   | Parallel-to-serial conversion |
| PIPO | Parallel | Parallel | Temporary parallel storage    |

### Interactive four-bit shift register

Choose the next serial input and press Clock.

0

0

0

0

#### Malayalam Support

ഓരോ clock pulse-യും data ഒരു flip-flop-ൽ നിന്ന് അടുത്ത flip-flop-ലേക്ക് shift ചെയ്യും. SISO serial input/serial output; SIPO serial input/parallel output.

## 6. Ring counter

A ring counter circulates a single 1 (or single 0) through n flip-flops. It normally provides n valid states and requires initialization.

n flip-flops → n ring-counter states

| Clock   | Q3Q2Q1Q0 |
|---------|----------|
| Initial | 1000     |
| 1       | 0100     |
| 2       | 0010     |
| 3       | 0001     |
| 4       | 1000     |

## 7. Johnson counter

A Johnson counter feeds the complemented output of the last flip-flop back to the first. An n-stage Johnson counter produces 2n valid states.

n flip-flops → 2n Johnson states

|   |      |
|---|------|
| 0 | 0000 |
| 1 | 1000 |
| 2 | 1100 |
| 3 | 1110 |
| 4 | 1111 |
| 5 | 0111 |
| 6 | 0011 |
| 7 | 0001 |
| 8 | 0000 |

Ring vs Johnson

Ring uses direct feedback and normally n states. Johnson uses complemented feedback and produces 2n states.

Module 3 expected exam questions

One mark

- Which circuits require clock input?
- Name SISO register.
- State Johnson states for n stages.

Three marks

- Draw D and T truth tables.
- Applications of shift registers.
- Draw 4-bit Johnson counter.

Seven marks

- JK-to-T and JK-to-D conversion.
- Explain PISO register.
- Explain ring or Johnson counter.

# Counters and Semiconductor Memories

Course 5044

This reconstructed chapter covers asynchronous and synchronous counter design, timing and memory types tested in the official model paper.

## Learning targets

- Explain ripple and synchronous counters.
- Design MOD-6, MOD-8 and MOD-10 counters.
- Trace up/down timing diagrams.
- Compare RAM, ROM, PROM, EPROM and EEPROM.

## Counter definition

A counter is a sequential circuit that advances through a prescribed state sequence in response to clock pulses.

## 1. MOD number and flip-flop requirement

Choose smallest  $n$  such that  $2^n \geq N$

$n = \text{ceiling}(\log_2 N)$

| Counter | Required flip-flops |
|---------|---------------------|
| MOD-6   | 3                   |
| MOD-8   | 3                   |
| MOD-10  | 4                   |
| MOD-16  | 4                   |

## 2. Asynchronous and synchronous counters

| Feature       | Asynchronous (ripple)                                                         | Synchronous                      |
|---------------|-------------------------------------------------------------------------------|----------------------------------|
| Clocking      | Only first FF receives external clock; others are clocked by preceding output | All FFs receive common clock     |
| Output change | Ripples stage by stage                                                        | Changes nearly simultaneously    |
| Speed         | Lower due to cumulative delay                                                 | Higher                           |
| Hardware      | Simpler                                                                       | More combinational control logic |
| Glitches      | More likely during transitions                                                | Reduced with proper design       |

## Diagram guidance

Draw the corresponding block or logic diagram with standard labels, signal direction and input/output names. Use the truth table and equations beside this panel to verify the drawing.



Ripple frequency division:  $f_{out} = f_{in}/2^n$

#### Malayalam Support

Ripple counter-ൽ clock change ഓരോ flip-flop വഴിയും വൈകി പടരുന്നു. Synchronous counter-ൽ എല്ലാ flip-flops-നും ഒരേ clock കിട്ടുന്നതിനാൽ state ഒരേസമയം മാറും.

### 3. Three-bit ripple up and down counters

A three-bit ripple up counter counts 000,001,010,011,100,101,110,111. A ripple down counter follows the reverse sequence. Whether Q or  $\bar{Q}$  clocks the next stage depends on flip-flop triggering convention.

| Decimal | Up Q2Q1Q0 | Down Q2Q1Q0 |
|---------|-----------|-------------|
| 0       | 000       | 111         |
| 1       | 001       | 110         |
| 2       | 010       | 101         |
| 3       | 011       | 100         |
| 4       | 100       | 011         |
| 5       | 101       | 010         |
| 6       | 110       | 001         |
| 7       | 111       | 000         |

### 4. Truncated asynchronous counters

A MOD-N counter with N not equal to a power of two uses reset logic to force the counter back to 000... after the Nth state.

#### MOD-6 design

1 N=6; choose 3 flip-flops because  $2^3=8 \geq 6$ .

2 Valid states: 000 through 101.

3 Detect 110 using reset/decode logic.

4 Clear all flip-flops to 000.

#### MOD-10 design

1  $N=10$ ; choose 4 flip-flops.

2 Valid states: 0000 through 1001.

3 Detect 1010.

4 Apply asynchronous clear to return to 0000.

## 5. Synchronous counter design

All flip-flops share the clock. The design procedure uses a state table, flip-flop excitation requirements, K-Maps and simplified input equations.

1 List present and next states.

2 Choose flip-flop type.

3 Use excitation table to find required inputs.

4 Minimize each input using K-Map.

5 Draw common-clock circuit and verify sequence.

3-bit synchronous up counter using T FFs:  $T_0=1$ ;  $T_1=Q_0$ ;  $T_2=Q_1Q_0$

3-bit synchronous down counter using T FFs:  $T_0=1$ ;  $T_1=\bar{Q}_0$ ;  $T_2=\bar{Q}_1\bar{Q}_0$

### Interactive 3-bit counter

0 0 0

Decimal state: 0

## 6. Semiconductor memories

A semiconductor memory stores binary words at numbered locations. Address lines select a location; data lines transfer a word.

Memory capacity = Number of locations  $\times$  bits per location

| Memory | Volatile? | Write method                      | Typical use                     |
|--------|-----------|-----------------------------------|---------------------------------|
| RAM    | Yes       | Electrical, repeated              | Temporary working data          |
| SRAM   | Yes       | Flip-flop cells; no refresh       | Cache / fast buffers            |
| DRAM   | Yes       | Capacitor cells; refresh required | Main memory                     |
| ROM    | No        | Fixed or programmed content       | Permanent firmware              |
| PROM   | No        | Programmed once                   | One-time configuration          |
| EPROM  | No        | UV erasure, then reprogram        | Older development systems       |
| EEPROM | No        | Electrical erase/rewrite          | Settings, calibration, firmware |

### Malayalam Support

RAM power off ചെയ്താൽ data നഷ്ടപ്പെടും; working data-കാണ്. ROM family power off ചെയ്താലും data നിലനിൽക്കും. PROM once, EPROM UV erase, EEPROM electrically erase ചെയ്യാം.

## 7. RAM vs ROM

| Point      | RAM                                      | ROM                               |
|------------|------------------------------------------|-----------------------------------|
| Nature     | Read/write memory                        | Primarily read memory             |
| Volatility | Volatile                                 | Non-volatile                      |
| Speed      | Generally faster for working storage     | Depends on technology             |
| Purpose    | Temporary data and programs in execution | Permanent instructions / firmware |

### One mark

- Name the synchronous counter.
- Flip-flops for MOD-10.
- Memory used for working data.

### Three marks

- Asynchronous vs synchronous.
- Types of RAM.
- RAM vs ROM.

### Seven marks

- Three-bit ripple up/down counter.
- MOD-6 or MOD-10 design.
- Synchronous up/down counter design.

## DIGITAL LOGIC BANKS

# Rules, formulas, symbols and design summaries

Use these compact banks for rapid revision and as a checklist while solving design problems.

## Formula and rule bank

1's complement: invert every bit

2's complement: 1's complement + 1

MUX:  $2^n$  inputs require n select lines

Counter FF count: smallest n with  $2^n \geq N$

Ripple division:  $f_{out} = f_{in} / 2^n$

Half adder:  $S = A \oplus B$ ;  $C = AB$

Full adder:  $S = A \oplus B \oplus C_{in}$ ;  $C_{out} = AB + AC_{in} + BC_{in}$

## Binary-to-Gray bank

$$G_n = B_n$$

$$G_i = B_{i+1} \oplus B_i$$

Copy the MSB. XOR each adjacent pair of binary bits for the remaining Gray bits.

## K-Map rule bank

- Groups: 1,2,4,8,16 cells.
- Use largest groups.
- Cover every required 1.
- Overlap if useful.
- Wrap-around allowed.
- No diagonal grouping.
- Don't-cares are optional.

## Memory capacity bank

$$\text{Capacity} = \text{locations} \times \text{bits per location}$$

Example:  $1K \times 8 = 1024 \text{ locations} \times 8 \text{ bits} = 8192 \text{ bits} = 1024 \text{ bytes}$ .

## Number-system conversion bank

| From             | To      | Method                                   |
|------------------|---------|------------------------------------------|
| Binary           | Decimal | Weighted sum of powers of 2              |
| Decimal integer  | Binary  | Repeated division by 2                   |
| Decimal fraction | Binary  | Repeated multiplication by 2             |
| Hex              | Binary  | Four bits per hex digit                  |
| Binary           | Hex     | Group bits in fours from radix point     |
| Decimal          | Hex     | Repeated division / multiplication by 16 |

Diagram guidance

Draw the corresponding block or logic diagram with standard labels, signal direction and input/output names. Use the truth table and equations beside this panel to verify the drawing.

| Gate | Boolean expression                    | Output 1 condition   |
|------|---------------------------------------|----------------------|
| AND  | $Y=AB$                                | All inputs 1         |
| OR   | $Y=A+B$                               | At least one input 1 |
| NOT  | $Y=\overline{A}$                      | Input 0              |
| NAND | $Y=(AB)\overline{\phantom{x}}$        | Except all inputs 1  |
| NOR  | $Y=(A+B)\overline{\phantom{x}}$       | All inputs 0         |
| XOR  | $Y=A\oplus B$                         | Inputs different     |
| XNOR | $Y=(A\oplus B)\overline{\phantom{x}}$ | Inputs equal         |

Combinational circuit bank

| Circuit        | Inputs / Outputs                              | Core equation or idea     |
|----------------|-----------------------------------------------|---------------------------|
| Half adder     | $A,B \rightarrow S,C$                         | $S=A\oplus B$ ; $C=AB$    |
| Full adder     | $A,B,C_{in} \rightarrow S,C_{out}$            | Two half adders + OR      |
| 4:1 MUX        | $I_0\text{--}I_3, S_1,S_0 \rightarrow Y$      | One selected input        |
| 1:4 DEMUX      | $D,S_1,S_0 \rightarrow Y_0\text{--}Y_3$       | One selected output       |
| Binary-to-Gray | $B_3\text{--}B_0 \rightarrow G_3\text{--}G_0$ | MSB direct, adjacent XORs |

Flip-flop and sequential bank

| Device | Hold      | Set   | Reset | Toggle / follow |
|--------|-----------|-------|-------|-----------------|
| SR     | $S=0,R=0$ | 1,0   | 0,1   | 1,1 invalid     |
| JK     | 0,0       | 1,0   | 0,1   | 1,1 toggles     |
| D      | —         | $D=1$ | $D=0$ | $Q_{next}=D$    |
| T      | $T=0$     | —     | —     | $T=1$ toggles   |

Timing diagram bank

When drawing timing diagrams, align vertical transition lines with the correct active clock edge. In a ripple counter, show delayed changes from Q0 to Q1 to Q2. In a synchronous counter, show all output changes referenced to the same clock edge.

Diagram guidance

Draw the corresponding block or logic diagram with standard labels, signal direction and input/output names. Use the truth table

## Counter design bank

| Task                | Essential steps                                                              |
|---------------------|------------------------------------------------------------------------------|
| Asynchronous MOD-N  | Choose FF count; list valid states; detect first invalid state; reset        |
| Synchronous counter | State table; excitation table; K-Maps; input equations; common-clock circuit |
| Up/down tracing     | Write state sequence and timing; note triggering convention                  |
| Frequency division  | Each binary stage divides frequency by 2                                     |

## Memory comparison bank

| Type   | Erase / program method           | Retains data without power? |
|--------|----------------------------------|-----------------------------|
| RAM    | Electrical read/write repeatedly | No                          |
| ROM    | Factory or fixed program         | Yes                         |
| PROM   | One-time electrical programming  | Yes                         |
| EPROM  | UV erase, electrical program     | Yes                         |
| EEPROM | Electrical erase and rewrite     | Yes                         |

### SOLVED EXAMPLES

## Exam-style worked problems

Each solution shows the method and final answer rather than only a hint.

### 1. Convert $125_{10}$ to hexadecimal

$125 \div 16 = 7$  remainder 13 (D)  
Read quotient and remainder: 7D  
Answer:  $(125)_{10} = (7D)_{16}$

### 2. Convert $(124.56)_{16}$ to decimal

Integer part:  $1 \times 16^2 + 2 \times 16 + 4 = 256 + 32 + 4 = 292$   
Fraction:  $5 \times 16^{-1} + 6 \times 16^{-2} = 5/16 + 6/256 = 0.3125 + 0.0234375 = 0.3359375$   
Answer:  $292.3359375_{10}$

### 3. Binary addition: $35 + 19$

$35_{10} = 100011_2$   
 $19_{10} = 010011_2$   
  100011  
+ 010011  
-----  
  110110  
Answer:  $54_{10} = 110110_2$

#### 4. Eight-bit two's-complement subtraction: 46–14

$$46 = 00101110$$

$$14 = 00001110$$

$$1\text{'s comp of } 14 = 11110001$$

$$2\text{'s comp} = 11110010$$

$$00101110 + 11110010 = 1\ 00100000$$

Discard carry.

$$\text{Answer: } 00100000_2 = 32_{10}$$

#### 5. Universal-gate conversion

NOT:  $A \text{ NAND } A = \bar{A}$

AND:  $A \text{ NAND } B$  gives  $(AB)^\top$ ; NAND that output with itself gives  $AB$ .

OR:  $A \text{ NAND } A$  gives  $\bar{A}$ ;  $A \text{ NAND } \bar{A}$  gives  $\bar{A}$ ;  $B \text{ NAND } B$  gives  $\bar{B}$ ;  $A \text{ NAND } \bar{B}$  gives  $A+B$  by De Morgan.

Thus NAND alone can implement the basic gates.

#### 6. Half-adder design

Truth table gives  $S=1$  for 01 and 10, so  $S=A \oplus B$ .

Carry is 1 only for 11, so  $C=AB$ .

Implement one XOR gate for Sum and one AND gate for Carry.

#### 7. Select lines for an 8:1 MUX

For  $2^n$  inputs, select lines= $n$ .

$$8=2^3, \text{ therefore } n=3.$$

Answer: 3 select lines.

#### 8. Four-bit binary to Gray

Input B3B2B1B0=1011

$$G3=B3=1$$

$$G2=B3 \oplus B2 = 1 \oplus 0 = 1$$

$$G1=B2 \oplus B1 = 0 \oplus 1 = 1$$

$$G0=B1 \oplus B0 = 1 \oplus 1 = 0$$

Answer: 1110 Gray

#### 9. JK to T conversion

$T=0$  requires hold; JK hold is  $J=K=0$ .

$T=1$  requires toggle; JK toggle is  $J=K=1$ .

Therefore  $J=T$  and  $K=T$ .

#### 10. Four-bit Johnson sequence

Start 0000. Feed complement of  $Q_3$  to  $Q_0$ .

$$0000 \rightarrow 1000 \rightarrow 1100 \rightarrow 1110 \rightarrow 1111 \rightarrow 0111 \rightarrow 0011 \rightarrow 0001 \rightarrow 0000.$$

A four-stage Johnson counter has  $2n=8$  states.

#### 11. Flip-flops for MOD-10

Find smallest  $n$  such that  $2^n \geq 10$ .

$$2^3=8 < 10; 2^4=16 \geq 10.$$

Answer: 4 flip-flops.



## 12 Memory capacity

A 2K×8 memory has 2048 locations and 8 bits/location.

Capacity=2048×8=16384 bits=2048 bytes=2 KB.

### MODULE-WISE EXPECTED QUESTIONS

## High-priority practice patterns

These are practice patterns inferred from the supplied model papers; they are not guaranteed examination questions.

### Module 1

#### Frequently tested

- Mixed conversion and complement problem.
- Universal gates and NAND implementation.
- Boolean laws or De Morgan.
- 3/4-variable K-Map with don't-cares.

### Module 2

#### Important design

- CMOS/ECL comparison and parameters.
- Half/full adder design.
- 4:1 MUX or 1:4 DEMUX.
- Binary-to-Gray converter.

### Module 3

#### Important diagram

- Flip-flop truth/function tables.
- JK-to-T or JK-to-D conversion.
- Shift-register operation.
- Ring/Johnson diagram and sequence.

### Module 4

#### Important timing

- Ripple up/down counter with timing.
- MOD-6/MOD-10 asynchronous design.
- Synchronous up/down design.
- RAM/ROM/PROM/EPROM/EEPROM comparison.

# Module-wise exercises and self-assessment

Attempt without looking at the answer key. Final answers are supplied below.

## Module 1 practice

1. Convert  $93_{10}$  to binary.
2. Convert  $3A7_{16}$  to decimal.
3. Find 8-bit two's complement of 00110101.
4. Subtract 27 from 52 using 8-bit two's complement.
5. Simplify  $F(A,B,C)=\Sigma m(1,3,5,7)$ .

## Module 2 practice

1. How many select lines for 16:1 MUX?
2. Write full-adder equations.
3. Design 2:1 MUX equation.
4. Convert binary 1101 to Gray.
5. Give three CMOS features.

## Module 3 practice

1. Complete JK table.
2. Show JK-to-D equations.
3. Trace 1011 through a four-stage SISO register.
4. List four shift-register applications.
5. Write 4-bit ring and Johnson sequences.

## Module 4 practice

1. Flip-flops required for MOD-12.
2. Write 3-bit up sequence.
3. State MOD-6 reset state.
4. Compare SRAM and DRAM.
5. Differentiate PROM, EPROM and EEPROM.

# Objective practice

Course 3044

| No. | Question                                           | Options                                              |
|-----|----------------------------------------------------|------------------------------------------------------|
| 1   | Logic family with traditionally least static power | A TTL • B CMOS • C ECL • D RTL                       |
| 2   | An 8:1 MUX requires                                | A 2 • B 3 • C 4 • D 8 select lines                   |
| 3   | Output depends on present input and past state in  | A combinational • B sequential • C decoder • D adder |
| 4   | Four-stage Johnson counter has                     | A 4 • B 6 • C 8 • D 16 states                        |
| 5   | Temporary working data is usually stored in        | A ROM • B RAM • C PROM • D EPROM                     |

## Practice answer key

**Module 1:** 1)  $1011101_2$ . 2)  $935_{10}$ . 3) 11001011. 4)  $25_{10} = 00011001_2$ . 5)  $F=C$ .

**Module 2:** 1) 4. 2)  $S=A \oplus B \oplus C_{in}$ ;  $C_{out}=AB+AC_{in}+BC_{in}$ . 3)  $Y=I_0\bar{S}+I_1S$ . 4) 1011. 5) Very low static power, high input impedance, good noise immunity.

**Module 3:** JK: hold/reset/set/toggle; JK-to-D  $J=D, K=\bar{D}$ ; remaining answers follow chapter tables.

**Module 4:** 4 FFs;  $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 110 \rightarrow 111$ ; reset when 110 appears; SRAM faster/no refresh, DRAM denser/refresh; PROM once, EPROM UV, EEPROM electrical.

**Objective:** 1-B, 2-B, 3-B, 4-C, 5-B.

## Self-assessment checklist

- ✓ I can show full number-conversion steps.
- ✓ I can draw all basic and universal gate symbols.
- ✓ I can group a four-variable K-Map correctly.
- ✓ I can derive adder, MUX and converter equations.
- ✓ I can trace flip-flop, register and counter states.
- ✓ I can compare memory types without mixing volatility and programming method.

## QUICK REVISION

# Last-day revision sheet

Use this section after completing the detailed chapters.

## Numbers

- Binary weights: powers of 2.
- Hex digit = 4 bits.
- 2's comp = invert + 1.

## Logic

- NAND/NOR universal.
- De Morgan swaps AND/OR and complements terms.
- XOR=inputs different.

## K-Maps

- Gray order 00,01,11,10.
- Largest power-of-two groups.
- Wrap yes; diagonal no.

## Combinational

- Half/full adder equations.
- $2^n$  MUX inputs need n selects.
- Gray uses adjacent XORs.

## Flip-flops

- SR set/reset.
- JK 11 toggles.
- D follows data.
- T 1 toggles.

## Registers

- SISO/SIPO/PISO/PIPO.
- One shift per clock.
- Ring=n states; Johnson= $2n$ .

## Counters

- Async ripple; sync common clock.
- Choose n with  $2^n \geq N$ .
- Each stage divides by 2.

## Memory

- RAM volatile.
- ROM non-volatile.

- PROM once; EPROM UV; EEPROM electrical.

Course 3044

### Common mistakes

Wrong bit width in complements; binary/hex grouping from wrong side; non-Gray K-Map ordering; diagonal grouping; missing carry term in full adder; confusing MUX with DEMUX; failing to initialize ring counter; wrong reset state in MOD counter; calling ROM volatile.

## FRESH MODEL QUESTION PAPER

# DIGITAL ELECTRONICS • Course 3044 • 3 Hours • 75 Marks

Fresh questions are based on the confirmed official structure and topic distribution; they are not copied from the supplied sets.

## PART A

Answer all questions in one word, one sentence or a small symbol. Each carries 1 mark. (9×1=9)

1. Write the binary equivalent of decimal 23.
2. Write the two's complement of 01011010.
3. Name the gate whose output is 1 when its two inputs are different.
4. Which logic family is traditionally noted for very high speed?
5. How many select lines are required for a 16:1 multiplexer?
6. Name the flip-flop whose next output directly follows its data input.
7. How many valid states are produced by a 5-stage Johnson counter?
8. How many flip-flops are required for a MOD-12 counter?
9. Name the non-volatile memory that can be erased electrically.

## PART B

Answer any eight. Each carries 3 marks. (8×3=24)

1. Convert  $(3B.8)_{16}$  to decimal.
2. State both De Morgan's theorems.
3. Implement NOT and AND using NAND gates only.
4. Define fan-out, propagation delay and power dissipation.
5. Write the truth table and expressions of a half adder.
6. Write the output equation and function table of a 2:1 multiplexer.
7. Write the truth tables of D and T flip-flops and one application of each.
8. List four applications of shift registers.
9. Give three differences between asynchronous and synchronous counters.
10. Compare SRAM and DRAM.

## PART C

Answer one alternative from every group. Each group carries 7 marks. (6×7=42)

1. **Module 1:** (A) Perform: (i)  $157_{10}$  to hexadecimal, (ii) 61–25 using 8-bit two's complement, (iii)  $7D3_{16}$  to binary. **OR** (B) Simplify  $F(A,B,C,D)=\Sigma m(0,2,5,7,8,10,13,15)$  using a K-Map.
2. **Module 2:** (A) Design a full adder from its truth table and realize it using two half adders. **OR** (B) Explain a 1:4 demultiplexer with function table, equations, logic diagram description and applications.
3. **Module 3:** (A) Convert a JK flip-flop into D and T flip-flops using conversion tables and equations. **OR** (B) Explain a four-bit Johnson counter with diagram description and state sequence.
4. **Module 3:** (A) Explain PISO and SIPO shift registers with operation tables. **OR** (B) Explain an SR flip-flop using NAND gates, including active-low truth table and invalid condition.
5. **Module 4:** (A) Explain a three-bit ripple up counter with logic and timing diagrams. **OR** (B) Compare RAM and ROM and explain PROM, EPROM and EEPROM.

## COMPLETE MODEL ANSWERS

# Full answers for every fresh-model question

Both alternatives in every Part C group are answered so students can study the complete pattern.

## Part A exact answers

| No. | Answer         |
|-----|----------------|
| 1   | $10111_2$      |
| 2   | 10100110       |
| 3   | XOR gate       |
| 4   | ECL            |
| 5   | 4 select lines |
| 6   | D flip-flop    |
| 7   | 10 states      |
| 8   | 4 flip-flops   |
| 9   | EEPROM         |

## Part B answers

### B1. Convert $(3B.8)_{16}$ to decimal

$$3 \times 16^1 + 11 \times 16^0 + 8 \times 16^{-1} = 48 + 11 + 0.5 = 59.5$$

$$\text{Answer: } (3B.8)_{16} = 59.5_{10}$$

### B2. De Morgan's theorems

$$\text{First: } (A+B)^- = \bar{A}\bar{B}$$

$$\text{Second: } (AB)^- = \bar{A} + \bar{B}$$

They are used to move complements across AND/OR operations while interchanging the operation.

### B3. NOT and AND using NAND

NOT: connect both NAND inputs to A, so  $Y = A \text{ NAND } A = \bar{A}$ .

AND: first  $X = (AB)^-$  using a NAND. Then  $Y = X \text{ NAND } X = \bar{X} = AB$ .

### B4. Logic-family parameters

Fan-out: number of standard gate inputs driven by one output.

Propagation delay: time between input transition and output response.

Power dissipation: average electrical power consumed by a gate, commonly measured in mW.

### B5. Half adder

Truth table:  $00 \rightarrow S0, C0$ ;  $01 \rightarrow 1, 0$ ;  $10 \rightarrow 1, 0$ ;  $11 \rightarrow 0, 1$ .

Sum  $S=A \oplus B$ ; Carry  $C=AB$ .

Use XOR for Sum and AND for Carry.

### B6. 2:1 multiplexer

Function:  $S=0$  selects  $I0$ ;  $S=1$  selects  $I1$ .

Equation:  $Y=I0\bar{S}+I1S$ .

The select line enables one AND path; an OR gate combines the paths.

### B7. D and T flip-flops

D:  $D=0 \rightarrow Q_{\text{next}}=0$ ;  $D=1 \rightarrow Q_{\text{next}}=1$ . Application: data register.

T:  $T=0 \rightarrow \text{hold}$ ;  $T=1 \rightarrow \text{toggle}$ . Application: binary counter / frequency divider.

### B8. Shift-register applications

Serial-to-parallel conversion, parallel-to-serial conversion, temporary data storage, digital delay line, sequence generation and data transfer. Any four earn full credit.

### B9. Asynchronous vs synchronous counters

Asynchronous: only first FF gets external clock; output ripples; cumulative delay makes it slower.

Synchronous: all FFs share common clock; outputs change together; faster but needs more control logic.

### B10. SRAM vs DRAM

SRAM uses bistable cells, is fast, needs no refresh and has lower density/higher cost.

DRAM uses capacitor cells, needs refresh, has higher density/lower cost and is used as main memory.

## Part C full answers

### C1A. Mixed number-system operations

i)  $157_{10}$  to hexadecimal:

$157 \div 16 = 9$  remainder 13(D)

Answer:  $9D_{16}$ .

ii) 61–25 using 8 bits:

$61 = 00111101$

$25 = 00011001$

1's complement of 25 =  $11100110$

2's complement =  $11100111$

$00111101$

+  $11100111$

-----

1  $00100100$

Discard carry.  $00100100_2 = 36_{10}$ .

iii)  $7D3_{16}$  to binary:

$7=0111$ ,  $D=1101$ ,  $3=0011$

Answer:  $0111\ 1101\ 0011_2$ .

### C1B. K-Map simplification

$F(A,B,C,D) = \sum m(0,2,5,7,8,10,13,15)$ .

Place 1s on the four-variable K-Map in Gray order.

Group  $m_0, m_2, m_8, m_{10}$  as a quad. In this group  $B=0$  and  $D=0$ , so term  $\overline{B}\overline{D}$ .

Group  $m_5, m_7, m_{13}, m_{15}$  as a quad. Here  $B=1$  and  $D=1$ , so term  $BD$ .

Final simplified expression:

$$F = \overline{B}\overline{D} + BD = B \text{ XNOR } D.$$

A and C disappear because they change inside each quad.

### C2A. Full-adder design

Inputs: A,B,Cin. Outputs: S,Cout.

Truth table:

$000 \rightarrow 00$ ,  $001 \rightarrow 10$ ,  $010 \rightarrow 10$ ,  $011 \rightarrow 01$ ,

$100 \rightarrow 10$ ,  $101 \rightarrow 01$ ,  $110 \rightarrow 01$ ,  $111 \rightarrow 11$ .

From K-Map:

$$S = A \oplus B \oplus \text{Cin}.$$

$$\text{Cout} = AB + AC_{\text{in}} + BC_{\text{in}}.$$

Realization using two half adders:

$$\text{HA1: } X = A \oplus B, \text{ C1} = AB.$$

$$\text{HA2: } S = X \oplus \text{Cin}, \text{ C2} = X \cdot \text{Cin}.$$

$$\text{Cout} = \text{C1} + \text{C2}.$$

This gives a complete full adder.

### C2B. 1:4 demultiplexer

A 1:4 DEMUX routes data input D to one of four outputs.

Function table:

$$S1S0=00 \rightarrow Y0=D$$

$$01 \rightarrow Y1=D$$

$$10 \rightarrow Y2=D$$

$$11 \rightarrow Y3=D$$

Equations:

$$Y0 = D \overline{S1} \overline{S0}$$

$$Y1 = D \overline{S1} S0$$

$$Y2 = D S1 \overline{S0}$$

$$Y3 = D S1 S0$$

Logic: two NOT gates generate complements; four 3-input AND gates produce outputs.

Applications: data routing, serial distribution, communication switching and memory selection.

### C3A. JK conversion to D and T

JK excitation:

$0 \rightarrow 0$  requires  $J=0, K=X$

$0 \rightarrow 1$  requires  $J=1, K=X$

$1 \rightarrow 0$  requires  $J=X, K=1$

$1 \rightarrow 1$  requires  $J=X, K=0$

$JK \rightarrow D$ :

Desired  $Q_{\text{next}} = D$ .

K-Map/excitation reduction gives  $J=D$  and  $K=\overline{D}$ .

Thus  $Q_{\text{next}}$  follows D.

$JK \rightarrow T$ :

$T=0$  requires hold, so  $J=K=0$ .

$T=1$  requires toggle, so  $J=K=1$ .

Therefore  $J=T$  and  $K=T$ .

Draw one inverter for K in  $JK \rightarrow D$  and direct T connections in  $JK \rightarrow T$ .



### C3B. Four-bit Johnson counter

Use four D flip-flops with common clock. Connect  $Q_0 \rightarrow D_1$ ,  $Q_1 \rightarrow D_2$ ,  $Q_2 \rightarrow D_3$  and feed  $\bar{Q}_3$  back to  $D_0$ .

Starting at 0000:

0000  $\rightarrow$  1000  $\rightarrow$  1100  $\rightarrow$  1110  $\rightarrow$  1111  $\rightarrow$  0111  $\rightarrow$  0011  $\rightarrow$  0001  $\rightarrow$  0000.

There are  $2n=8$  valid states.

Because complemented feedback is used, it is called a twisted-ring counter.

Applications: timing sequence generation, divide-by-8 sequence and non-overlapping control signals.

### C4A. PISO and SIPO registers

SIPO:

Data enters serially into first flip-flop. Each clock shifts bits one stage. After four clocks,  $Q_3, Q_2, Q_1, Q_0$  provide the parallel word.

Example serial input 1,0,1,1 from reset:

Clock1 1000; Clock2 0100; Clock3 1010; Clock4 1101 (orientation depends on drawing).

PISO:

Parallel inputs are loaded simultaneously using LOAD control. In SHIFT mode, each clock transfers one stored bit to serial output.

Applications: communication interfaces and reducing wire count.

Diagrams should show four flip-flops, common clock, serial path and load/shift selection.

### C4B. SR latch using NAND

The cross-coupled NAND latch has active-low  $\bar{S}$  and  $\bar{R}$ .

$\bar{S}$   $\bar{R}$ :

1 1  $\rightarrow$  hold

0 1  $\rightarrow$  set  $Q=1$

1 0  $\rightarrow$  reset  $Q=0$

0 0  $\rightarrow$  invalid because both outputs become 1 and the final state after release is uncertain.

Two NAND outputs are cross-coupled, giving memory by positive feedback.

Never apply both active-low inputs at 0 simultaneously.

### C5A. Three-bit ripple up counter

Use three T flip-flops with  $T=1$ . External clock drives FF0.  $Q_0$  clocks FF1 and  $Q_1$  clocks FF2 according to the chosen edge convention.

State sequence:

000,001,010,011,100,101,110,111, then 000.

$Q_0$  frequency= $f_{in}/2$ ;  $Q_1=f_{in}/4$ ;  $Q_2=f_{in}/8$ .

In timing diagram  $Q_0$  toggles every clock,  $Q_1$  every two clocks and  $Q_2$  every four clocks.

Because each stage responds after the previous one, transitions ripple and cumulative delay occurs.

### C5B. RAM, ROM, PROM, EPROM and EEPROM

RAM is volatile read/write memory for temporary working data.

ROM is non-volatile memory used for fixed programs or firmware.

PROM is programmed once by the user.

EPROM can be erased using ultraviolet light and reprogrammed.

EEPROM can be erased and rewritten electrically.

RAM is used during active computation; ROM-family devices retain information without power.

### C6A. MOD-6 asynchronous counter

Required modulus  $N=6$ .

Flip-flops: smallest  $n$  with  $2^n \geq 6$  is  $n=3$ .

Use three T flip-flops with  $T=1$  as a ripple counter.

Valid states: 000,001,010,011,100,101.

The next natural state is 110. Decode  $Q_2=1$  and  $Q_1=1$  (state 110) using a NAND/AND reset network according to active-clear polarity.

Apply asynchronous clear to all flip-flops, forcing 000.

Unused states 110 and 111 are eliminated by reset. Draw three T FFs, ripple clock connections and reset decoder.

### C6B. Three-bit synchronous down counter using T flip-flops

All T flip-flops receive the same clock.

Down sequence: 111,110,101,100,011,010,001,000, then 111.

T input requirements:

$T_0=1$  because  $Q_0$  toggles every clock.

$T_1=\bar{Q}_0$  because  $Q_1$  toggles when lower bit  $Q_0$  is 0.

$T_2=\bar{Q}_1\bar{Q}_0$  because  $Q_2$  toggles when both lower bits are 0.

Connect these equations using NOT and AND gates. Since all flip-flops share the clock, state changes are synchronous.

#### REFERENCE LEARNING LINKS

## Supplementary resources

The supplied source does not specify official textbooks. These links are supplementary and are not claimed as official course references.

### Official model-paper page

SITTTTR model question-paper page for course 3044

### NPTEL

NPTEL course portal — search for Digital Circuits / Digital Electronics.

### All About Circuits

Digital textbook for supplementary reading.

#### FINAL EXAMINATION CHECKLIST

# Before you submit the answer book

Course 3044

## Calculations

- ✓ Base and bit width shown.
- ✓ Complement steps shown.
- ✓ Carry treatment shown.
- ✓ Final answer labelled with base.

## Design answers

- ✓ Truth/state table included.
- ✓ K-Map groups marked.
- ✓ Reduced equations written.
- ✓ Diagram labelled and operation explained.

### Final exam tip

For a seven-mark design question, a bare final circuit is not enough. Include the table, reduction, equations, circuit and working explanation.